Análisis

El programa simula la misión del satélite Quetzal-2. En ella, los paneles solares producen energía, los módulos de vuelo recolectan datos científicos y los módulos de tierra reciben esos datos. Todos participan en el mismo ciclo de la misión, pero cada tipo realiza una tarea diferente.

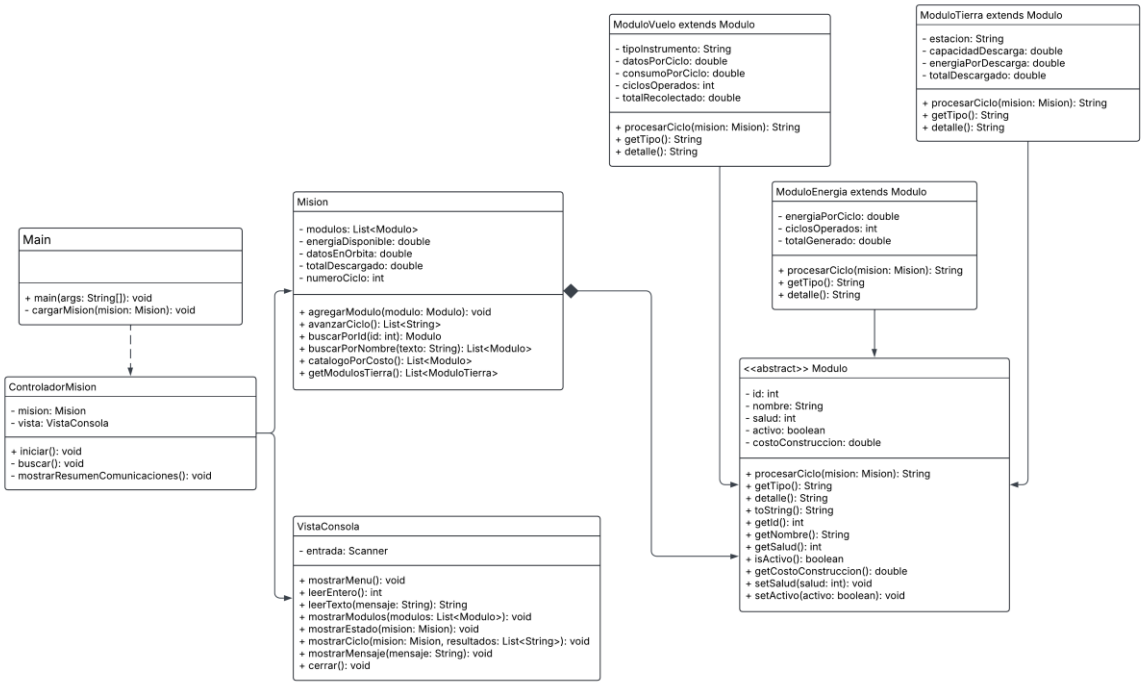
Para resolver el problema, la misión conserva los módulos en una sola lista y los procesa en el orden en que aparecen. Al iniciar cada ciclo, el primer módulo tiene su oportunidad de trabajar. Después, el siguiente usa los recursos que quedaron disponibles. Por ejemplo, un panel puede producir energía que luego consume un módulo de vuelo; si el vuelo recolecta datos, una antena que participe después puede descargarlos. Si un módulo está inactivo o no cuenta con los recursos necesarios, el programa informa la razón y continúa con el siguiente.

La carga inicial incluye diez módulos y comienza con 50 unidades de energía y sin datos en órbita. El menú permite consultar los módulos, buscar uno, ordenar el catálogo por costo, avanzar ciclos y revisar el resumen de comunicaciones. La información y los recursos se conservan entre ciclos.

Descripción de las clases

- **Modulo** es la clase general para representar cualquier pieza de la misión. Guarda id, nombre, salud, activo y costoConstruccion. Define la operación procesarCiclo(), que cada tipo concreto realiza a su manera, y métodos para mostrar su tipo y sus detalles.
- **ModuloVuelo** representa una cámara o un sensor que recolecta datos. Incluye tipoInstrumento, datosPorCiclo, consumoPorCiclo, ciclosOperados y totalRecolectado. Solo recolecta si está activo y hay suficiente energía; al hacerlo, agrega datos pendientes a la misión.
- **ModuloTierra** representa una antena que descarga datos de órbita. Guarda estacion, capacidadDescarga, energiaPorDescarga y totalDescargado. Descarga hasta su capacidad, sin exceder los datos disponibles, y consume la energía establecida para cada descarga.
- **ModuloEnergia** representa un panel solar. Tiene energiaPorCiclo, ciclosOperados y totalGenerado. Si está activo, agrega energía a la misión.
- **Mision** administra la lista de módulos y los recursos compartidos. Guarda energiaDisponible, datosEnOrbita, totalDescargado y numeroCiclo. También avanza ciclos, busca módulos y prepara el catálogo ordenado por costo.

- **VistaConsola** presenta el menú y los resultados al operador, y lee sus respuestas.
- **ControladorMision** coordina las opciones del menú: recibe la selección desde la vista y solicita a la misión la acción correspondiente.
- **Main** inicia el programa, crea la misión, agrega los módulos iniciales y pone en marcha el controlador.


ModuloVuelo extends Modulo
- tipoInstrumento: String
- datosPorCiclo: double
- consumoPorCiclo: double
- ciclosOperados: int
- totalRecolectado: double
+ procesarCiclo(mision: Mision): String
+ getTipo(): String
+ detalle(): String
ModuloTierra extends Modulo
- estacion: String
- capacidadDescarga: double
- energiaPorDescarga: double
- totalDescargado: double
+ procesarCiclo(mision: Mision): String
+ getTipo(): String
+ detalle(): String
Mision
- modulos: List<Modulo>
- energiaDisponible: double
- datosEnOrbita: double
- totalDescargado: double
- numeroCiclo: int
+ agregarModulo(modulo: Modulo): void
+ avanzarCiclo(): List<String>
+ buscarPorId(id: int): Modulo
+ buscarPorNombre(texto: String): List<Modulo>
+ catalogoPorCosto(): List<Modulo>
+ getModulosTierra(): List<ModuloTierra>
ModuloEnergia extends Modulo
- energiaPorCiclo: double
- ciclosOperados: int
- totalGenerado: double
+ procesarCiclo(mision: Mision): String
+ getTipo(): String
+ detalle(): String
Main
+ main(args: String[]): void
- cargarMision(mision: Mision): void
ControladorMision
- mision: Mision
- vista: VistaConsola
+ iniciar(): void
- buscar(): void
- mostrarResumenComunicaciones(): void
<<abstract>> Modulo
- id: int
- nombre: String
- salud: int
- activo: boolean
- costoConstruccion: double
+ procesarCiclo(mision: Mision): String
+ getTipo(): String
+ detalle(): String
+ toString(): String
+ getId(): int
+ getNombre(): String
+ getSalud(): int
+ isActivo(): boolean
+ getCostoConstruccion(): double
+ setSalud(salud: int): void
+ setActivo(activo: boolean): void
VistaConsola
- entrada: Scanner
+ mostrarMenu(): void
+ leerEntero(): int
+ leerTexto(mensaje: String): String
+ mostrarModulos(modulos: List<Modulo>): void
+ mostrarEstado(mision: Mision): void
+ mostrarCiclo(mision: Mision, resultados: List<String>): void
+ mostrarMensaje(mensaje: String): void
+ cerrar(): void